

One Person Lab

OPL App 白皮书

让复杂知识工作拥有一个本地优先、结果可复查、可以持续推进的专业工作台

一个专业 AI 产品是否好用，不取决于它暴露了多少模型、智能体和运行参数，而取决于用户能否从真实目标出发，得到有来路的成果，知道何时继续、何时判断，并在工作环境变化后仍能接着做。

目的优先 / 成果有来路 / 状态指向行动 / 工作位置可连续

2026-07-13

Public whitepaper

目录

1	OPL App 解决的不是一次问答	2
2	设计一：从工作目的开始	2
3	设计二：成果必须带着来路	2
4	设计三：状态必须指向行动	3
5	设计四：工作位置可以变化，工作语言不变	3
6	一个完整场景：从本机研究到团队审阅	3
7	只维护三个软件对象	4
8	清楚的分工带来可信的产品	4
9	本文边界	4
10	结语	5

发布日期：2026-07-13

适用对象：希望理解 OPL App 为什么这样设计，以及这些设计怎样帮助自己完成研究、基金、演示和书稿等复杂知识工作的用户、合作者、早期采用者和技术决策者。

核心判断：一个专业 AI 产品是否好用，不取决于它暴露了多少模型、智能体和运行参数，而取决于用户能否从真实目标出发，得到有来路的成果，知道何时继续、何时判断，并在工作环境变化后仍能接着做。

1 OPL App 解决的不是一次问答

AI 已经很擅长回答问题、生成文字和修改文件。真正困难的是把这些能力放进一项会持续数天、数周甚至数月的正式工作。

一篇论文会经历材料整理、研究问题收敛、分析、作图、写作、审阅和修订；一份基金申请会反复调整创新点、技术路线和论证；一套演示要让数据、故事线、视觉和导出结果彼此一致；一本书更需要跨章节保持结构、风格和事实连续。

在这些工作里，用户需要的不只是“这一轮回答得怎么样”，而是：

- 我现在应该从哪里开始？
- 当前成果由哪些材料、分析和审阅支持？
- 工作推进到了哪里，下一步可以自动继续还是需要我决定？
- 离开几天后回来，是否还能接住之前的上下文？
- 换到浏览器、服务器或云端时，是否仍在同一条工作线上？

OPL App 因此不是又一个聊天框，也不是把内部系统搬到屏幕上的 agent dashboard。它是 One Person Lab 面向用户的工作台：让目标、对话、材料、阶段、产物、审阅、回执和下一步形成一条可以继续的工作线。

2 设计一：从工作目的开始

用户打开 OPL App，首先应该表达“我要完成什么”，而不是先选择仓库、provider、shell 或某个内部缩写。

默认首页保留四个高频工作目的：

用户看到的入口	用户想完成的事	默认专业 Package
科研	从问题、材料和数据走向分析、证据与论文交付	MAS
基金	形成有说服力的选题、论证、申请书与修订	MAG
演示	把内容、数据和故事线变成可审阅的视觉交付	RCA
写书	组织大纲、章节、风格、修订与出版交接	OBF

这四个入口是首屏的默认起点，不是 OPL App 的能力上限。专业能力以 **OPL Agent Packages** 进入工作台：用户可以安装、发现和管理更多合规 Package，并决定哪些快捷入口出现在首页。首页保持简单，能力体系保持可扩展，两者不必互相牺牲。

OPL Meta Agent (OMA) 同样是可用的专业 Package，但它服务于“设计或改进智能体”这一较少发生、需要更多上下文的工作，因此不占用默认首页。需要它的用户可以从 Packages 能力面进入，也可以把它显式加入首页。这个安排不是降低 OMA 的地位，而是让日常入口服从用户频率和工作目的。

这样的设计有两个好处：新用户不必先学习 One Person Lab 的组织结构，熟练用户也不会被固定菜单限制。产品可以不断增加专业能力，首屏仍然只回答一个问题：你今天要完成什么？

3 设计二：成果必须带着来路

正式知识工作的结果不能只是一份文件。一个结论、一张图、一页演示或一个章节，只有能回到它所依赖的材料、过程和审阅，才值得被继续使用。

OPL App 把工作表达为一条可以复查的链：

目标 -> 材料与资源 -> 阶段推进 -> 产物 -> 审阅与检查 -> 回执 -> 下一步

“结果有来路”并不意味着把工程日志全部倒给用户。用户首先看到的是与判断有关的信息：当前产物在哪里，哪些内容已经检查，哪里仍有质量债，下一步是继续修订、补充材料、等待资源，还是由人作决定。只有需要排查问题时，路径、标识、原始回执和运行细节才进入诊断层。

这条原则让 App 能同时避免两种风险：一是只给最终文件，让用户无法判断是否可信；二是把所有内部证据铺在首屏，让用户在技术细节中找不到真正要做的事。

OPL App 负责把已有事实翻译成用户能采取行动的状态，但不制造第二套事实。运行状态来自 OPL Base，Package 状态来自 Framework 管理的 Package 记录，研究、基金、视觉和书稿质量来自对应专业 Agent 与人工 owner。界面可以说明“发生了什么”和“下一步是什么”，不能凭一次渲染替任何 owner 宣布 ready。

4 设计三：状态必须指向行动

复杂产品很容易把设置页变成内部结构目录：运行时、集成、技能、缓存、路径、回执和 provider 各占一栏。这样的界面看似完整，却把理解系统的成本转嫁给用户。

OPL App 采用相反的组织方式。设置首先围绕用户要做的决定：

- 当前环境是否可以开始工作，需要我做什么？
- App、Base 或 Package 是否需要安装、更新或修复？
- 哪些 Agent Packages 已可用，哪些显示在首页？
- 浏览器访问、远端资源或账号能力是否需要配置？
- 哪些数据由本机持有，清理或迁移会影响什么？

默认层只显示目的、状态、影响和主动作。Package id、manifest、路径、缓存、底层回执和完整 ledger 等信息折叠到“高级诊断”，供需要排障的人按需展开。

这种信息架构建立了一条稳定规则：**正常状态保持安静，异常状态说清影响，所有状态都给出合法下一步**。用户不需要成为运维人员，仍然能知道该继续工作、等待后台完成、批准资源，还是进入修复。

5 设计四：工作位置可以变化，工作语言不变

OPL App 是本地优先的。用户可以先在自己的 Mac 上选择工作目录，让敏感材料、早期草稿和个人项目留在自己的环境里。需要 Linux、Windows、服务器或远程访问时，同一产品体验可以通过 Docker/WebUI 在浏览器中承载；当浏览器工作台由云端托管，并接入账号、存储、隔离和资源策略时，它成为 OPL Workspace。

本机 OPL App

- > 本地或服务器上的 Docker/WebUI
- > 托管的 OPL Workspace

连续的不是某一个安装包，而是用户理解工作的语言：项目、任务、阶段、产物、审阅、阻塞、回执和下一步。用户不应因为运行位置变化，就重新学习一套产品或丢失成果的来路。

这是一项产品设计承诺，不是对所有载体当前版本的发布声明。桌面 App、Docker/WebUI 和 OPL Workspace 是否在某个版本达到相同功能、真实数据连续、稳定发布或 production ready，必须分别由 release artifact、运行回读、端到端 evidence 和 owner acceptance 证明。白皮书、合同通过或界面截图都不能替代这些证据。

6 一个完整场景：从本机研究到团队审阅

设想一位研究者准备一篇需要重新分析数据的论文。

从目标开始。她在本机打开 OPL App，选择自己的项目目录和“科研”入口，用普通语言说明研究问题、数据位置和希望形成的交付物。App 将任务交给已安装的 MAS Package，而不是要求她手动拼接模型、技能和命令。

让过程可见。MAS 围绕问题、数据、分析、图表、论断和稿件推进。App 不用一张“正在运行”的卡片遮住整个过程，而是持续呈现当前阶段、最新产物、需要补充的材料和等待她决定的问题。她可以打开结果，也可以回到支持结果的材料和审阅意见。

在正确的时刻让人判断。如果分析已经形成可消费的增量，但某张图仍需重画，工作可以带着清楚标注的质量债继续；如果要声称结果达到投稿标准，则必须回到 MAS 的质量门和负责人确认。App 展示这一区别，不自行把“有进展”改写为“已就绪”。

按需改变工作位置。当本机资源不足时，她可以批准使用服务器或云端资源；需要合作者参与时，可以在有相应运行与发布证据的浏览器或 Workspace 环境继续。项目仍以相同方式呈现任务、产物、审阅和下一步，而资源绑定、账号和组织策略留在对应管理面。

把下一轮接住。一周后回来，她看到的不是一段需要重新阅读的长对话，而是当前稿件、最近一次审阅、未关闭的问题和可以继续的动作。产品的价值不在于替她作最终学术判断，而在于让专业判断有材料、有上下文、有清楚的交接点。这个场景也适用于基金、演示和书稿：专业内容不同，可信工作的结构相同。

7 只维护三个软件对象

用户不应该为了使用专业 AI 而理解一套内部依赖图。OPL 把日常安装、更新和修复收束为三个软件对象：

对象	用户应如何理解
OPL Base	没有界面的运行基础，负责让任务、阶段、Package 和回执可靠工作
OPL App	用户看到和操作的工作台，负责入口、交互、状态翻译与产品体验
OPL Packages	可安装和管理的专业 Agent、能力与工作流，决定“能完成哪类专业工作”

运行时、Codex 投影、工作流 profile、companion tools 和底层集成都是这三个对象内部的状态，不再变成额外 updater。用户数据与产物则拥有独立的存储、保留和清理边界，它们不是第四个软件对象。

这个模型让维护动作更容易理解：App 的更新不会暗中改写 Base 或 Packages；Package 的安装与修复有自己的记录和回滚引用；界面展示维护状态，但 Framework 仍持有 Package 与运行事实。

8 清楚的分工带来可信的产品

OPL App 不单独完成所有工作。它的专业性恰恰来自清楚分工：

- 用户目标
- > OPL App：入口、交互、状态与下一步

-> OPL Base：阶段运行、恢复、投影与回执

-> OPL Packages：领域判断、审阅与交付

-> Workspace / Cloud：按需提供托管、资源与组织能力

App 不替 Framework 定义运行真相，不替 MAS、MAG、RCA、OBF 或 OMA 作领域质量裁决，也不替云端管理面决定账号、资源和组织策略。它把这些 owner 已经持有的事实组织成一个普通用户可以理解和操作的工作台。

边界清楚不是工程洁癖。它直接影响用户信任：状态不会因为换一个界面就改口，专业结论不会因为一张绿色卡片就被提前宣布，资源和权限也不会在用户不知情时越界。

9 本文边界

本文解释 OPL App 的设计理念和用户价值。它不是安装说明、设置手册、功能清单、发布证明、运行状态证明或领域质量证明。

当前安装和使用方式以 App 的公开 README 与用户指南为准；版本、签名、更新通道和 release readiness 以 release artifacts、updater metadata、CI evidence 与 owner decision 为准；运行状态以 OPL Framework 的 CLI/API 和 runtime readback 为准；领域质量以对应专业 Agent 和人工 owner 为准。

10 结语

OPL App 希望让一个人拥有类似专业团队的工作界面，而不是再增加一个需要照料的复杂系统。

它让用户从真实目的开始，让成果带着来路，让状态指向下一步，让设置围绕用户决策组织；它允许工作从本机走向浏览器和云端，但不把设计目标包装成已经完成的发布事实；它把能力放进可扩展 Packages，同时让日常维护只保留 Base、App 和 Packages 三个对象。

产品好用不是因为复杂性消失了，而是因为复杂性被放在正确的位置。用户看到目标、成果和决定，专业 Agent 保留判断权，运行系统保留事实，诊断细节在需要时出现。这就是 OPL App 的设计选择，也是它值得被信任的原因。